Penetration Testing Sandbox Lab Report

Target System: MockVulnerable E-Shop (Flask/SQLite Application)
Assessment Date: October 2026
Status: Post-Remediation Verification Complete

1. Executive Summary

This security assessment evaluates the operational risk and structural vulnerability profile of the **MockVulnerable E-Shop** platform. Initial exploratory scans identified **Critical-severity vulnerability items** that allowed full administrative authentication bypass and unauthorized cross-tenant data harvesting. By implementing parameterized SQL bindings and rigorous cryptographic multi-tenant state checks, the security layer successfully neutralized automated attack workflows, restoring the system's baseline integrity.

2. Scope and Authorization

The testing boundary was strictly confined to the localized sandbox container environment operating at http://127.0.0.1:5000. Full operational authorization was established for simulated hostile traffic execution, client-side session manipulation, and multi-tenant credential harvesting simulations.

3. Tools and Methodology

The evaluation utilized a hybrid testing approach combining manual browser injection exercises with automated script execution using the Python requests framework (via exploit.py). Vulnerability identification benchmarks were aligned with the **OWASP Web Security Testing Guide (WSTG)** framework rules.

4. Results and Evidence

During initial configuration runs, the automated execution engine successfully manipulated database structural commands. The payload admin' -- returned an unauthenticated administrator session token instantly. Concurrently, modifying URL endpoint arguments to /receipt?order_id=2 dumped records belonging to secondary users without system blockages.

5. Findings with Severity and Remediation

Finding	Severity	Remediation Strategy
SQL Injection (SQLi) via Login Portal	**CRITICAL**	Deploy parameterized SQLite statement placeholders (? bindings) to decouple input parsing from executable data logic.
Insecure Direct Object References (IDOR)	**HIGH**	Enforce backend authorization identity cross-checks by filtering order query structures directly against session['user_id'].

6. Limitations

This assessment represents a controlled simulation restricted to local execution vectors. It does not account for complex multi-layered environments, third-party edge firewall infrastructure, or advanced distributed web denial-of-service patterns.

7. Conclusion and Recommended Next Steps

While the deployed code-level patches completely resolve SQL Injection and IDOR vulnerabilities, long-term application security requires continuous mitigation efforts. Recommended next steps include implementing cryptographic password hashing routines (e.g., via Bcrypt or Werkzeug tools) and standardizing global role-based access control (RBAC) definitions across all operational dashboards.